RAJALAKSHMI ENGINEERING COLLEGE

Abdul Hameed A 251001003-2025-IT

A2

| | |
|---|---|
| **Started on** | Friday, 31 July 2026, 3:41 PM |
| **State** | Finished |
| **Completed on** | Saturday, 8 August 2026, 9:05 PM |
| **Time taken** | 8 days 5 hours |
| **Marks** | 1.00/1.00 |
| **Grade** | **10.00** out of 10.00 (**100**%) |

**Question 1** | Correct Mark 1.00 out of 1.00

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void function (int n)
{
    int i= 1;
```

```
    int s =1;
```

```
    while(s <= n)
    {
         i++;
         s += i;
     }
}
```

**Note:** No need of counter increment for declarations and scanf() and count variable printf() statements.

**Input:**

A positive Integer n

**Output:**

Print the value of the counter variable

**For example:**

| Input | Result |
|---|---|
| 9 | 12 |

**Answer:** (penalty regime: 0 %)

```
#include<stdio.h>
int main()
{
int n,count;
count=0;
scanf("%d",&n);
int i=1;
count++;
int s=1;
count++;
while(s<=n)
{
count++;
i++;
count++;
s+=i;
count++;
}
count++;
printf("%d",count);

}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 9 | 12 | 12 | ✔ |
| ✔ | 4 | 9 | 9 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Abdul Hameed A 251001003-2025-IT

A2

| | |
|---|---|
| **Started on** | Friday, 31 July 2026, 4:26 PM |
| **State** | Finished |
| **Completed on** | Saturday, 8 August 2026, 9:03 PM |
| **Time taken** | 8 days 4 hours |
| **Marks** | 1.00/1.00 |
| **Grade** | **10.00** out of 10.00 (**100**%) |

**Question 1** | Correct Mark 1.00 out of 1.00

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void func(int n)
{
    if(n==1)
    {
      printf("*");
    }
    else
    {
     for(int i=1; i<=n; i++)
     {
       for(int j=1; j<=n; j++)
       {
          printf("*");
          printf("*");
          break;
       }
     }
   }
 }
```

**Note:** No need of counter increment for declarations and scanf() and count variable printf() statements.

**Input:**

A positive Integer n

**Output:**

Print the value of the counter variable

**Answer:** (penalty regime: 0 %)

```
#include<stdio.h>
int main()
{
    int c=0;
    int n;
    scanf("%d",&n);
    if(n==1)
    {
        c++;
        c++;
    }
    else
    {
        c++;
        for(int i=1;i<=n;i++)
        {
            c++;
            for(int j=1;j<=n;j++)
            {
            c++;
            c++;
            c++;
            break;
            }
            c++;
        }
        c++;
    }
    printf("%d",c);
}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 2 | 12 | 12 | ✔ |
| ✔ | 1000 | 5002 | 5002 | ✔ |
| ✔ | 143 | 717 | 717 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Abdul Hameed A 251001003-2025-IT

A2

| | |
|---|---|
| **Started on** | Friday, 31 July 2026, 4:43 PM |
| **State** | Finished |
| **Completed on** | Saturday, 8 August 2026, 9:02 PM |
| **Time taken** | 8 days 4 hours |
| **Marks** | 1.00/1.00 |
| **Grade** | **10.00** out of 10.00 (**100**%) |

**Question 1** | Correct Mark 1.00 out of 1.00

Convert the following algorithm into a program and find its time complexity using counter method.

```
Factor(num) {
{
   for (i = 1; i <= num;++i)
   {
    if (num % i== 0)
       {
         printf("%d ", i);
       }
    }
 }
```

**Note:** No need of counter increment for declarations and scanf() and counter variable printf() statement.

**Input:**

A positive Integer n

**Output:**

Print the value of the counter variable

**Answer:**

```
#include <stdio.h>

void Factor(int num) {
    int c=0;
    for(int i=1;i<=num;++i)
    {
        c++;
        c++;
        if(num%i==0)
        {
            c++;
        }
    }
    c++;
    printf("%d",c);
}
int main()
{
    int n;
    scanf("%d",&n);
    Factor(n);
}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 12 | 31 | 31 | ✔ |
| ✔ | 25 | 54 | 54 | ✔ |
| ✔ | 4 | 12 | 12 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Abdul Hameed A 251001003-2025-IT

A2

| | |
|---|---|
| **Started on** | Saturday, 8 August 2026, 8:12 PM |
| **State** | Finished |
| **Completed on** | Saturday, 8 August 2026, 9:02 PM |
| **Time taken** | 49 mins 26 secs |
| **Marks** | 1.00/1.00 |
| **Grade** | **10.00** out of 10.00 (**100**%) |

**Question 1** | Correct Mark 1.00 out of 1.00

Convert the following algorithm into a program and find its time

complexity using counter method.

```
void function(int n)
{
    int c= 0;
    for(int i=n/2; i<n; i++)
        for(int j=1; j<n; j = 2 * j)
            for(int k=1; k<n; k = k * 2)
                c++;
}
```

**Note:** No need of counter increment for declarations and scanf() and count variable printf() statements.

**Input:**

A positive Integer n

**Output:**

Print the value of the counter variable

**Answer:**

```
#include<stdio.h>
void function(int n)
{
    int counter=0;
    int c= 0;
    counter++;
    for(int i=n/2; i<n; i++)
    {
        counter++;
        for(int j=1; j<n; j = 2 * j)
        {
            counter++;
             for(int k=1; k<n; k = k * 2)
             {
             counter++;
             c++;
             counter++;
             }
             counter++;
        }
        counter++;
    }
    counter++;
    printf("%d",counter);
}
int main()
{
    int n;
    scanf("%d",&n);
    function(n);

}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 4 | 30 | 30 | ✔ |
| ✔ | 10 | 212 | 212 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Abdul Hameed A 251001003-2025-IT

A2

| | |
|---|---|
| **Started on** | Saturday, 8 August 2026, 8:24 PM |
| **State** | Finished |
| **Completed on** | Saturday, 8 August 2026, 9:02 PM |
| **Time taken** | 37 mins 12 secs |
| **Marks** | 1.00/1.00 |
| **Grade** | **10.00** out of 10.00 (**100**%) |

**Question 1** | Correct Mark 1.00 out of 1.00

Convert the following algorithm into a program and find its time complexity using counter method.

```
void reverse(int n)
{
   int rev = 0, remainder;
   while (n != 0)
    {
        remainder = n % 10;
        rev = rev * 10 + remainder;
        n/= 10;

    }
print(rev);
}
```

**Note:** No need of counter increment for declarations and scanf() and count variable printf() statements.

**Input:**

A positive Integer n

**Output:**

Print the value of the counter variable

**Answer:**

```c
#include<stdio.h>
void reverse(int n)
{
   int c=0;
   int rev = 0, remainder;
   c++;
   while (n != 0)
    {
        c++;
        remainder = n % 10;
        c++;
        rev = rev * 10 + remainder;
        c++;
        n/= 10;
        c++;
    }
    c++;

    c++;
    printf("%d",c);
}
int main()
{
    int n;
    scanf("%d",&n);
    reverse(n);
}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 12 | 11 | 11 | ✔ |
| ✔ | 1234 | 19 | 19 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Abdul Hameed A 251001003-2025-IT **A2**

| | |
|---|---|
| **Started on** | Thursday, 27 August 2026, 10:08 AM |
| **State** | Finished |
| **Completed on** | Thursday, 27 August 2026, 10:09 AM |
| **Time taken** | 1 min 7 secs |
| **Marks** | 1.00/1.00 |
| **Grade** | **10.00** out of 10.00 (**100**%) |

**Question 1** | Correct Mark 1.00 out of 1.00

Write a program to take value V and we want to make change for V Rs, and we have infinite supply of each of the denominations in Indian currency, i.e., we have infinite supply of { 1, 2, 5, 10, 20, 50, 100, 500, 1000} valued coins/notes, what is the minimum number of coins and/or notes needed to make the change.

Input Format:

Take an integer from stdin.

Output Format:

print the integer which is change of the number.

Example Input :

64

Output:

4

Explanaton:

We need a 50 Rs note and a 10 Rs note and two 2 rupee coins.

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>

int main() {
    int V;
    scanf("%d", &V);

    int denominations[] = {1000, 500, 100, 50, 20, 10, 5, 2, 1};
    int count = 0;
    int i;

    for (i = 0; i < 9; i++) {
        count += V / denominations[i];
        V = V % denominations[i];
    }

    printf("%d\n", count);

    return 0;
}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 49 | 5 | 5 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Abdul Hameed A 251001003-2025-IT **A2**

| | |
|---|---|
| **Started on** | Thursday, 27 August 2026, 10:09 AM |
| **State** | Finished |
| **Completed on** | Thursday, 27 August 2026, 10:10 AM |
| **Time taken** | 52 secs |
| **Marks** | 1.00/1.00 |
| **Grade** | **10.00** out of 10.00 (**100**%) |

**Question 1** | Correct Mark 1.00 out of 1.00

Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.

Each child i has a greed factor g[i], which is the minimum size of a cookie that the child will be content with; and each cookie j has a size s[j]. If s[j] >= g[i], we can assign the cookie j to the child i, and the child i will be content. Your goal is to maximize the number of your content children and output the maximum number.

**Example 1:**

**Input:**

3

1 2 3

2

1 1

**Output:**

1

Explanation: You have 3 children and 2 cookies. The greed factors of 3 children are 1, 2, 3.

And even though you have 2 cookies, since their size is both 1, you could only make the child whose greed factor is 1 content.

You need to output 1.

**Constraints:**

1 <= g.length <= 3 * 10^4

0 <= s.length <= 3 * 10^4

1 <= g[i], s[j] <= 2^31 - 1

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

int compare(const void *a, const void *b) {
    return (*(int *)a > *(int *)b) - (*(int *)a < *(int *)b);
}

int main() {
    int n, m;

    // Number of children
    scanf("%d", &n);

    int g[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &g[i]);
    }

    // Number of cookies
    scanf("%d", &m);

    int s[m];
    for (int i = 0; i < m; i++) {
        scanf("%d", &s[i]);
    }

    // Sort greed factors and cookie sizes
    qsort(g, n, sizeof(int), compare);
    qsort(s, m, sizeof(int), compare);

    int child = 0;
    int cookie = 0;

    // Greedy matching
    while (child < n && cookie < m) {
        if (s[cookie] >= g[child]) {
            child++;
        }
```

```
39          cookie++;
40      }
41
42      printf("%d\n", child);
43
44      return 0;
45  }
46
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 2<br>1 2<br>3<br>1 2 3 | 2 | 2 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Abdul Hameed A 251001003-2025-IT **A2**

| | |
|---|---|
| **Started on** | Thursday, 27 August 2026, 10:11 AM |
| **State** | Finished |
| **Completed on** | Sunday, 30 August 2026, 7:40 PM |
| **Time taken** | 3 days 9 hours |
| **Marks** | 1.00/1.00 |
| **Grade** | **10.00** out of 10.00 (**100**%) |

**Question 1** | Correct Mark 1.00 out of 1.00

A person needs to eat burgers. Each burger contains a count of calorie. After eating the burger, the person needs to run a distance to burn out his calories.

If he has eaten i burgers with c calories each, then he has to run at least $3^i * c$ kilometers to burn out the calories. For example, if he ate 3

burgers with the count of calorie in the order: [1, 3, 2], the kilometers he needs to run are $(3^0 * 1) + (3^1 * 3) + (3^2 * 2) = 1 + 9 + 18 = 28$.

But this is not the minimum, so need to try out other orders of consumption and choose the minimum value. Determine the minimum distance

he needs to run. Note: He can eat burger in any order and use an efficient sorting algorithm.Apply greedy approach to solve the problem.

**Input Format**

First Line contains the number of burgers

Second line contains calories of each burger which is n space-separate integers

**Output Format**

Print: Minimum number of kilometers needed to run to burn out the calories

**Sample Input**

3

5 10 7

**Sample Output**

76

**For example:**

| Test | Input | Result |
|---|---|---|
| Test Case 1 | 3<br>1 3 2 | 18 |

**Answer:** (penalty regime: 0 %)

```
#include<stdio.h>
#include<math.h>
int main(){
    int n;
    scanf("%d",&n);
    int arr[n];
    for(int i=0;i<n;i++){
        scanf("%d",&arr[i]);
    }
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] < arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
    int s=0;
    for(int i=0;i<n;i++){
        s+=(pow(n,i))*arr[i];
    }
    printf("%d",s);
}
```

| | Test | Input | Expected | Got | |
|---|---|---|---|---|---|
| ✔ | Test Case 1 | 3<br>1 3 2 | 18 | 18 | ✔ |
| ✔ | Test Case 2 | 4<br>7 4 9 6 | 389 | 389 | ✔ |
| ✔ | Test Case 3 | 3<br>5 10 7 | 76 | 76 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Abdul Hameed A 251001003-2025-IT **A2**

| | |
|---|---|
| **Started on** | Thursday, 27 August 2026, 10:17 AM |
| **State** | Finished |
| **Completed on** | Thursday, 27 August 2026, 10:18 AM |
| **Time taken** | 55 secs |
| **Marks** | 1.00/1.00 |
| **Grade** | **10.00** out of 10.00 (**100**%) |

**Question 1** | Correct Mark 1.00 out of 1.00

Given an array of N integer, we have to maximize the sum of arr[i] * i, where i is the index of the element (i = 0, 1, 2, ..., N).Write an algorithm based on Greedy technique with a Complexity O(nlogn).

Input Format:

First line specifies the number of elements-n

The next n lines contain the array elements.

Output Format:

Maximum Array Sum to be printed.

Sample Input:

5

2 5 3 4 0

Sample output:

40

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

int compare(const void *a, const void *b) {
    int x = *(int *)a;
    int y = *(int *)b;

    return (x > y) - (x < y);
}

int main() {
    int n;
    scanf("%d", &n);

    long long arr[n];

    for (int i = 0; i < n; i++) {
        scanf("%lld", &arr[i]);
    }

    // Sort in ascending order
    qsort(arr, n, sizeof(long long), compare);

    long long sum = 0;

    for (int i = 0; i < n; i++) {
        sum += arr[i] * i;
    }

    printf("%lld\n", sum);

    return 0;
}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 5<br>2<br>5<br>3<br>4<br>0 | 40 | 40 | ✔ |

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 10<br>2<br>2<br>2<br>4<br>4<br>3<br>3<br>5<br>5<br>5 | 191 | 191 | ✔ |
| ✔ | 2<br>45<br>3 | 45 | 45 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Abdul Hameed A 251001003-2025-IT **A2**

| | |
|---|---|
| **Started on** | Thursday, 27 August 2026, 10:18 AM |
| **State** | Finished |
| **Completed on** | Thursday, 27 August 2026, 10:29 AM |
| **Time taken** | 10 mins 35 secs |
| **Marks** | 1.00/1.00 |
| **Grade** | **10.00** out of 10.00 (**100**%) |

**Question 1** | Correct Mark 1.00 out of 1.00

Given two arrays array_One[] and array_Two[] of same size N. We need to first rearrange the arrays such that the sum of the product of pairs( 1 element from each) is minimum. That is SUM (A[i] * B[i]) for all i is minimum.

**For example:**

| Input | Result |
|---|---|
| 3<br>1<br>2<br>3<br>4<br>5<br>6 | 28 |

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

int ascending(const void *a, const void *b) {
    int x = *(int *)a;
    int y = *(int *)b;
    return (x > y) - (x < y);
}

int descending(const void *a, const void *b) {
    int x = *(int *)a;
    int y = *(int *)b;
    return (y > x) - (y < x);
}

int main() {
    int n;
    scanf("%d", &n);

    long long array_One[n];
    long long array_Two[n];

    for (int i = 0; i < n; i++) {
        scanf("%lld", &array_One[i]);
    }

    for (int i = 0; i < n; i++) {
        scanf("%lld", &array_Two[i]);
    }

    // Sort first array in ascending order
    qsort(array_One, n, sizeof(long long), ascending);

    // Sort second array in descending order
    qsort(array_Two, n, sizeof(long long), descending);

    long long sum = 0;

    for (int i = 0; i < n; i++) {
        sum += array_One[i] * array_Two[i];
    }

    printf("%lld\n", sum);

    return 0;
}

```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 3<br>1<br>2<br>3<br>4<br>5<br>6 | 28 | 28 | ✔ |
| ✔ | 4<br>7<br>5<br>1<br>2<br>1<br>3<br>4<br>1 | 22 | 22 | ✔ |
| ✔ | 5<br>20<br>10<br>30<br>10<br>40<br>8<br>9<br>4<br>3<br>10 | 590 | 590 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Abdul Hameed A 251001003-2025-IT **A2**

| | |
|---|---|
| **Started on** | Thursday, 6 August 2026, 11:25 AM |
| **State** | Finished |
| **Completed on** | Thursday, 6 August 2026, 11:26 AM |
| **Time taken** | 40 secs |
| **Marks** | 1.00/1.00 |
| **Grade** | **10.00** out of 10.00 (**100**%) |

**Question 1** | Correct Mark 1.00 out of 1.00

**Problem Statement**

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format

First Line Contains Integer m – Size of array

Next m lines Contains m numbers – Elements of an array

Output Format

First Line Contains Integer – Number of zeroes present in the given array.

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>

// Divide and Conquer function to find the index of the first occurrence of 0
int countZeros(int arr[], int low, int high, int m) {
    if (high >= low) {
        int mid = low + (high - low) / 2;

        // Check if mid is the first occurrence of 0
        // It is the first 0 if it's at index 0 or the element before it is 1
        if ((mid == 0 || arr[mid - 1] == 1) && arr[mid] == 0) {
            return m - mid; // Total size - starting index of 0 gives count of 0s
        }

        // If element is 1, the first 0 must be on the right side
        if (arr[mid] == 1) {
            return countZeros(arr, mid + 1, high, m);
        }
        // Otherwise, the first 0 is on the left side
        else {
            return countZeros(arr, low, mid - 1, m);
        }
    }

    // If no 0 is found (all elements are 1)
    return 0;
}

int main() {
    int m;
    if (scanf("%d", &m) != 1) return 0;

    int arr[m];
    for (int i = 0; i < m; i++) {
        scanf("%d", &arr[i]);
    }

    // Special case: if the first element is 1 and last is 1, there are 0 zeros.
    // If the first element is 0, all elements are zeros.
    if (arr[0] == 0) {
        printf("%d\n", m);
    } else if (arr[m - 1] == 1) {
        printf("0\n");
    } else {
        int zeros = countZeros(arr, 0, m - 1, m);
        printf("%d\n", zeros);
    }

    return 0;
}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 5<br>1<br>1<br>1<br>0<br>0 | 2 | 2 | ✔ |
| ✔ | 10<br>1<br>1<br>1<br>1<br>1<br>1<br>1<br>1<br>1<br>1 | 0 | 0 | ✔ |
| ✔ | 8<br>0<br>0<br>0<br>0<br>0<br>0<br>0<br>0 | 8 | 8 | ✔ |
| ✔ | 17<br>1<br>1<br>1<br>1<br>1<br>1<br>1<br>1<br>1<br>1<br>1<br>1<br>1<br>1<br>1<br>0<br>0 | 2 | 2 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Abdul Hameed A 251001003-2025-IT **A2**

| | |
|---|---|
| **Started on** | Thursday, 6 August 2026, 11:26 AM |
| **State** | Finished |
| **Completed on** | Thursday, 6 August 2026, 11:26 AM |
| **Time taken** | 27 secs |
| **Marks** | 1.00/1.00 |
| **Grade** | **10.00** out of 10.00 (**100**%) |

**Question 1** | Correct Mark 1.00 out of 1.00

Given an array `nums` of size `n`, return *the majority element*.

The majority element is the element that appears more than `⌊n / 2⌋` times. You may assume that the majority element always exists in the array.

**Example 1:**

```
Input: nums = [3,2,3]
Output: 3
```

**Example 2:**

```
Input: nums = [2,2,1,1,1,2,2]
Output: 2
```

**Constraints:**

- `n == nums.length`
- `1 <= n <= 5 * 10^4`
- `-2^31 <= nums[i] <= 2^31 - 1`

**For example:**

| Input | Result |
|---|---|
| 3<br>3 2 3 | 3 |
| 7<br>2 2 1 1 1 2 2 | 2 |

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>

// Function using Boyer-Moore Voting Algorithm to find the majority element
int majorityElement(int nums[], int n) {
    int candidate = 0;
    int count = 0;

    for (int i = 0; i < n; i++) {
        if (count == 0) {
            candidate = nums[i];
            count = 1;
        } else if (nums[i] == candidate) {
            count++;
        } else {
            count--;
        }
    }

    return candidate;
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int nums[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &nums[i]);
    }

    int result = majorityElement(nums, n);
    printf("%d\n", result);

    return 0;
```

```
35 }
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 3<br>3 2 3 | 3 | 3 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Abdul Hameed A 251001003-2025-IT **A2**

| | |
|---|---|
| **Started on** | Thursday, 6 August 2026, 11:26 AM |
| **State** | Finished |
| **Completed on** | Thursday, 6 August 2026, 11:27 AM |
| **Time taken** | 28 secs |
| **Marks** | 1.00/1.00 |
| **Grade** | **10.00** out of 10.00 (**100**%) |

**Question 1** | Correct Mark 1.00 out of 1.00

**Problem Statement:**

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

**Input Format**

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Value for x

**Output Format**

First Line Contains Integer – Floor value for x

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>

// Divide and Conquer / Binary Search approach to find the floor of x
int findFloor(int arr[], int low, int high, int x) {
    // If low exceeds high, no floor exists (return -1 or handle appropriately)
    if (low > high) {
        return -1;
    }

    // If x is greater than or equal to the last element, then the last element is the floor
    if (x >= arr[high]) {
        return arr[high];
    }

    // If x is smaller than the first element, no floor exists in this range
    if (x < arr[low]) {
        return -1;
    }

    int mid = low + (high - low) / 2;

    // If element is exact match, it is the floor
    if (arr[mid] == x) {
        return arr[mid];
    }

    // If mid element is smaller than x, then mid could be a potential floor,
    // but we should check if there's a larger element on the right side.
    if (arr[mid] < x) {
        int floorVal = findFloor(arr, mid + 1, high, x);
        return (floorVal != -1) ? floorVal : arr[mid];
    }

    // If mid element is greater than x, the floor must lie in the left half
    return findFloor(arr, low, mid - 1, x);
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int x;
    scanf("%d", &x);

    int floor = findFloor(arr, 0, n - 1, x);
    if (floor != -1) {
        printf("%d\n", floor);
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 6<br>1<br>2<br>8<br>10<br>12<br>19<br>5 | 2 | 2 | ✔ |
| ✔ | 5<br>10<br>22<br>85<br>108<br>129<br>100 | 85 | 85 | ✔ |
| ✔ | 7<br>3<br>5<br>7<br>9<br>11<br>13<br>15<br>10 | 9 | 9 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Abdul Hameed A 251001003-2025-IT **A2**

| | |
|---|---|
| **Started on** | Thursday, 6 August 2026, 11:24 AM |
| **State** | Finished |
| **Completed on** | Thursday, 6 August 2026, 11:25 AM |
| **Time taken** | 59 secs |
| **Marks** | 1.00/1.00 |
| **Grade** | **10.00** out of 10.00 (**100**%) |

**Question 1** | Correct Mark 1.00 out of 1.00

**Problem Statement:**

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as "No".

Note: Write a Divide and Conquer Solution

**Input Format**

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Sum Value

**Output Format**

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value "x")

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdbool.h>

// Recursive function using divide and conquer (binary search approach)
// to find if an element 'target' exists in arr[low...high]
bool binarySearch(int arr[], int low, int high, int target, int *foundVal) {
    if (low > high)
        return false;

    int mid = low + (high - low) / 2;
    if (arr[mid] == target) {
        *foundVal = arr[mid];
        return true;
    }
    else if (arr[mid] > target) {
        return binarySearch(arr, low, mid - 1, target, foundVal);
    }
    else {
        return binarySearch(arr, mid + 1, high, target, foundVal);
    }
}

// Recursive divide and conquer function to check pairs
bool findPair(int arr[], int low, int high, int x, int *num1, int *num2) {
    if (low >= high)
        return false;

    // Divide step: pick the middle element and search for (x - arr[mid]) in the rest
    int mid = low + (high - low) / 2;
    int complement = x - arr[mid];
    int foundVal = -1;

    // Search for complement using binary search (excluding mid itself)
    // We can search either in [low, mid-1] or [mid+1, high] depending on complement value
    if (complement < arr[mid]) {
        if (binarySearch(arr, low, mid - 1, complement, &foundVal)) {
            *num1 = foundVal;
            *num2 = arr[mid];
            return true;
        }
    } else {
        if (binarySearch(arr, mid + 1, high, complement, &foundVal)) {
            *num1 = arr[mid];
            *num2 = foundVal;
            return true;
        }
    }

    // Conquer step: recursively check left and right halves
    if (findPair(arr, low, mid - 1, x, num1, num2))
        return true;

```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 4<br>2<br>4<br>8<br>10<br>14 | 4<br>10 | 4<br>10 | ✔ |
| ✔ | 5<br>2<br>4<br>6<br>8<br>10<br>100 | No | No | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Abdul Hameed A 251001003-2025-IT **A2**

| | |
|---|---|
| **Started on** | Thursday, 13 August 2026, 10:23 AM |
| **State** | Finished |
| **Completed on** | Wednesday, 30 September 2026, 3:15 PM |
| **Time taken** | 48 days 4 hours |
| **Marks** | 1.00/1.00 |
| **Grade** | **10.00** out of 10.00 (**100**%) |

**Question 1** | Correct Mark 1.00 out of 1.00

Write a Program to Implement the Quick Sort Algorithm

Input Format:
The first line contains the no of elements in the list-n
The next n lines contain the elements.

Output:
Sorted list of elements

**For example:**

| Input | Result |
|---|---|
| 5<br>67 34 12 98 78 | 12 34 67 78 98 |

**Answer:**

```c
#include <stdio.h>

void quickSort(int a[], int low, int high) {
    if (low < high) {
        int i = low, j = high;
        int pivot = a[low];
        int temp;

        while (i < j) {
            while (a[i] <= pivot && i < high)
                i++;

            while (a[j] > pivot)
                j--;

            if (i < j) {
                temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }

        a[low] = a[j];
        a[j] = pivot;

        quickSort(a, low, j - 1);
        quickSort(a, j + 1, high);
    }
}

int main() {
    int n, i;
    int a[100];

    scanf("%d", &n);

    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    quickSort(a, 0, n - 1);

    for (i = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}

```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 5<br>67 34 12 98 78 | 12 34 67 78 98 | 12 34 67 78 98 | ✔ |
| ✔ | 10<br>1 56 78 90 32 56 11 10 90 114 | 1 10 11 32 56 56 78 90 90 114 | 1 10 11 32 56 56 78 90 90 114 | ✔ |
| ✔ | 12<br>9 8 7 6 5 4 3 2 1 10 11 90 | 1 2 3 4 5 6 7 8 9 10 11 90 | 1 2 3 4 5 6 7 8 9 10 11 90 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Abdul Hameed A 251001003-2025-IT

A2

| | |
|---|---|
| **Started on** | Friday, 11 September 2026, 3:56 PM |
| **State** | Finished |
| **Completed on** | Friday, 11 September 2026, 4:41 PM |
| **Time taken** | 44 mins 52 secs |
| **Grade** | **10.00** out of 10.00 (**100**%) |

**Question 1** | Correct Mark 10.00 out of 10.00

**Playing with Numbers:**

Ram and Sita are playing with numbers by giving puzzles to each other. Now it was Ram term, so he gave Sita a positive integer 'n' and two numbers 1 and 3. He asked her to find the possible ways by which the number n can be represented using 1 and 3.Write any efficient algorithm to find the possible ways.

**Example 1:**

***Input:*** *6*
***Output:****6*
***Explanation:*** *There are 6 ways to 6 represent number with 1 and 3*

*1+1+1+1+1+1*
*3+3*
*1+1+1+3*
*1+1+3+1*
*1+3+1+1*
*3+1+1+1*

**Input Format**

First Line contains the number n

**Output Format**

**Print: The number of possible ways 'n' can be represented using 1 and 3**

Sample Input

6

Sample Output

6

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    long long dp[n + 1];

    for (int i = 0; i <= n; i++)
        dp[i] = 0;

    dp[0] = 1;

    for (int i = 1; i <= n; i++) {
        dp[i] += dp[i - 1];
        if (i >= 3)
            dp[i] += dp[i - 3];
    }

    printf("%lld", dp[n]);

    return 0;
}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 6 | 6 | 6 | ✔ |

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 25 | 8641 | 8641 | ✔ |
| ✔ | 100 | 24382819596721629 | 24382819596721629 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 10.00/10.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Abdul Hameed A 251001003-2025-IT

A2

| | |
|---|---|
| **Started on** | Friday, 11 September 2026, 4:41 PM |
| **State** | Finished |
| **Completed on** | Friday, 11 September 2026, 4:42 PM |
| **Time taken** | 37 secs |
| **Grade** | **10.00** out of 10.00 (**100**%) |

**Question 1** | Correct Mark 10.00 out of 10.00

**Playing with Chessboard:**

Ram is given with an n*n chessboard with each cell with a monetary value. Ram stands at the (0,0), that the position of the top left white rook. He is been given a task to reach the bottom right black rook position (n-1, n-1) constrained that he needs to reach the position by traveling the maximum monetary path under the condition that he can only travel one step right or one step down the board. Help ram to achieve it by providing an efficient DP algorithm.

**Example:**

**Input**

3

**1** 2 4

**2** 3 4

**8 7 1**

**Output:**

19

**Explanation:**

Totally there will be 6 paths among that the optimal is

Optimal path value:1+2+8+7+1=19

**Input Format**

First Line contains the integer n

The next n lines contain the n*n chessboard values

**Output Format**

Print Maximum monetary value of the path

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int a[n][n], dp[n][n];

    // Read the chessboard values
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &a[i][j]);
        }
    }

    // Starting cell
    dp[0][0] = a[0][0];

    // First row
    for (int j = 1; j < n; j++) {
        dp[0][j] = dp[0][j - 1] + a[0][j];
    }

    // First column
    for (int i = 1; i < n; i++) {
        dp[i][0] = dp[i - 1][0] + a[i][0];
    }

    // Fill the remaining cells
    for (int i = 1; i < n; i++) {
        for (int j = 1; j < n; j++) {
            if (dp[i - 1][j] > dp[i][j - 1])
                dp[i][j] = dp[i - 1][j] + a[i][j];
            else
                dp[i][j] = dp[i][j - 1] + a[i][j];
        }
```

```
37      }
38
39      // Print the maximum monetary value
40      printf("%d", dp[n - 1][n - 1]);
41
42      return 0;
43  }
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 3<br>1 2 4<br>2 3 4<br>8 7 1 | 19 | 19 | ✔ |
| ✔ | 3<br>1 3 1<br>1 5 1<br>4 2 1 | 12 | 12 | ✔ |
| ✔ | 4<br>1 1 3 4<br>1 5 7 8<br>2 3 4 6<br>1 6 9 0 | 28 | 28 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 10.00/10.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Abdul Hameed A 251001003-2025-IT

A2

| | |
|---|---|
| **Started on** | Thursday, 24 September 2026, 10:17 AM |
| **State** | Finished |
| **Completed on** | Thursday, 24 September 2026, 10:20 AM |
| **Time taken** | 3 mins 12 secs |
| **Marks** | 1.00/1.00 |
| **Grade** | **10.00** out of 10.00 (**100**%) |

**Question 1** | Correct Mark 1.00 out of 1.00

Given two strings find the length of the common longest subsequence(need not be contiguous) between the two.

Example:

s1: ggtabe

s2: tgatasb

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| s1 | | a | g | **g** | **t** | **a** | **b** |
| s2 | **g** | x | **t** | x | **a** | **y** | **b** |

**The length is 4**

Solveing it using Dynamic Programming

**For example:**

| Input | Result |
|---|---|
| aab<br>azb | 2 |

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

// Helper function to find the maximum of two integers
int max(int a, int b) {
    return (a > b) ? a : b;
}

// Function to find the length of the LCS
int longestCommonSubsequence(char *s1, char *s2) {
    int m = strlen(s1);
    int n = strlen(s2);

    // Dynamically allocate a 2D DP array of size (m + 1) x (n + 1)
    int **dp = (int **)malloc((m + 1) * sizeof(int *));
    for (int i = 0; i <= m; i++) {
        dp[i] = (int *)calloc(n + 1, sizeof(int)); // calloc initializes elements to 0
    }

    // Fill the DP table bottom-up
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (s1[i - 1] == s2[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1] + 1;
            } else {
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
    }

    int result = dp[m][n];

    // Free the dynamically allocated memory to prevent memory leaks
    for (int i = 0; i <= m; i++) {
        free(dp[i]);
    }
    free(dp);

    return result;
}

int main() {
    char s1[1005], s2[1005];

    // Read pairs of strings from standard input until End Of File (EOF)
    // scanf automatically handles whitespace and newlines between strings
    while (scanf("%s %s", s1, s2) == 2) {
        printf("%d\n", longestCommonSubsequence(s1, s2));
    }
```

```
51
52      return 0;
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | aab<br>azb | 2 | 2 | ✔ |
| ✔ | ABCD<br>ABCD | 4 | 4 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Abdul Hameed A 251001003-2025-IT

A2

| | |
|---|---|
| **Started on** | Thursday, 24 September 2026, 10:21 AM |
| **State** | Finished |
| **Completed on** | Thursday, 24 September 2026, 10:21 AM |
| **Time taken** | 34 secs |
| **Marks** | 1.00/1.00 |
| **Grade** | **10.00** out of 10.00 (**100**%) |

**Question 1** | Correct Mark 1.00 out of 1.00

Problem statement:

Find the length of the Longest Non-decreasing Subsequence in a given Sequence.

Eg:

Input:9

Sequence:[-1,3,4,5,2,2,2,2,3]

the subsequence is [-1,2,2,2,2,3]

Output:6

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

int longestNonDecreasingSubsequence(int arr[], int n) {
    if (n == 0) return 0;

    // Allocate memory for the DP array
    int *dp = (int *)malloc(n * sizeof(int));
    int max_len = 1;

    for (int i = 0; i < n; i++) {
        dp[i] = 1; // Base case: minimum length is 1
        for (int j = 0; j < i; j++) {
            // Condition for non-decreasing subsequence
            if (arr[i] >= arr[j]) {
                if (dp[j] + 1 > dp[i]) {
                    dp[i] = dp[j] + 1;
                }
            }
        }
        // Track the overall maximum length
        if (dp[i] > max_len) {
            max_len = dp[i];
        }
    }

    free(dp);
    return max_len;
}

int main() {
    int n;

    // Read the size of the sequence
    if (scanf("%d", &n) == 1) {
        int *arr = (int *)malloc(n * sizeof(int));

        // Read the sequence elements
        for (int i = 0; i < n; i++) {
            scanf("%d", &arr[i]);
        }

        // Compute and print the result
        printf("%d\n", longestNonDecreasingSubsequence(arr, n));

        free(arr);
    }

    return 0;
}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 9<br>-1 3 4 5 2 2 2 2 3 | 6 | 6 | ✔ |
| ✔ | 7<br>1 2 2 4 5 7 6 | 6 | 6 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Abdul Hameed A 251001003-2025-IT

A2

| | |
|---|---|
| **Started on** | Thursday, 24 September 2026, 10:22 AM |
| **State** | Finished |
| **Completed on** | Thursday, 24 September 2026, 10:23 AM |
| **Time taken** | 38 secs |
| **Marks** | 1.00/1.00 |
| **Grade** | **10.00** out of 10.00 (**100**%) |

**Question 1** | Correct Mark 1.00 out of 1.00

Find Duplicate in Array.

Given a read only array of n integers between 1 and n, find one number that repeats.

Input Format:

First Line - Number of elements

n Lines - n Elements

Output Format:

Element x - That is repeated

**For example:**

| Input | Result |
|---|---|
| 5<br>1 1 2 3 4 | 1 |

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

int findDuplicate(int arr[], int n) {
    int low = 1, high = n;

    while (low < high) {
        int mid = low + (high - low) / 2;
        int count = 0;

        // Count how many elements are less than or equal to mid
        for (int i = 0; i < n; i++) {
            if (arr[i] <= mid) {
                count++;
            }
        }

        // If count exceeds mid, the duplicate is in the lower half
        if (count > mid) {
            high = mid;
        } else {
            // Otherwise, it's in the upper half
            low = mid + 1;
        }
    }

    return low;
}

int main() {
    int n;

    // Read number of elements
    if (scanf("%d", &n) == 1) {
        int *arr = (int *)malloc(n * sizeof(int));

        // Read the array elements
        for (int i = 0; i < n; i++) {
            scanf("%d", &arr[i]);
        }

        // Find and print the duplicate element
        printf("%d\n", findDuplicate(arr, n));

        free(arr);
    }

    return 0;
}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 11<br>10 9 7 6 5 1 2 3 8 4 7 | 7 | 7 | ✔ |
| ✔ | 5<br>1 2 3 4 4 | 4 | 4 | ✔ |
| ✔ | 5<br>1 1 2 3 4 | 1 | 1 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Abdul Hameed A 251001003-2025-IT

A2

| | |
|---|---|
| **Started on** | Thursday, 24 September 2026, 10:23 AM |
| **State** | Finished |
| **Completed on** | Thursday, 24 September 2026, 10:24 AM |
| **Time taken** | 53 secs |
| **Marks** | 1.00/1.00 |
| **Grade** | **10.00** out of 10.00 (**100**%) |

**Question 1** | Correct Mark 1.00 out of 1.00

Find Duplicate in Array.

Given a read only array of n integers between 1 and n, find one number that repeats.

Input Format:

First Line - Number of elements

n Lines - n Elements

Output Format:

Element x - That is repeated

**For example:**

| Input | Result |
|---|---|
| 5<br>1 1 2 3 4 | 1 |

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

int findDuplicate(const int *arr, int n) {
    int low = 1, high = n - 1;

    while (low < high) {
        int mid = low + (high - low) / 2;
        int count = 0;

        // Count how many elements are less than or equal to mid
        for (int i = 0; i < n; i++) {
            if (arr[i] <= mid) {
                count++;
            }
        }

        // If count exceeds mid, the duplicate is in the lower half
        if (count > mid) {
            high = mid;
        } else {
            low = mid + 1;
        }
    }

    return low;
}

int main() {
    int n;

    // Read the number of elements
    if (scanf("%d", &n) == 1) {
        int *arr = (int *)malloc(n * sizeof(int));

        // Read the n elements
        for (int i = 0; i < n; i++) {
            scanf("%d", &arr[i]);
        }

        // Find and print the repeated element
        int duplicate = findDuplicate(arr, n);
        printf("%d\n", duplicate);

        free(arr);
    }

    return 0;
}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 11<br>10 9 7 6 5 1 2 3 8 4 7 | 7 | 7 | ✔ |
| ✔ | 5<br>1 2 3 4 4 | 4 | 4 | ✔ |
| ✔ | 5<br>1 1 2 3 4 | 1 | 1 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Abdul Hameed A 251001003-2025-IT

A2

| | |
|---|---|
| **Started on** | Thursday, 24 September 2026, 10:24 AM |
| **State** | Finished |
| **Completed on** | Thursday, 24 September 2026, 10:24 AM |
| **Time taken** | 30 secs |
| **Marks** | 1.00/1.00 |
| **Grade** | **10.00** out of 10.00 (**100**%) |

**Question 1** | Correct Mark 1.00 out of 1.00

Find the intersection of two sorted arrays.

OR in other words,

Given 2 sorted arrays, find all the elements which occur in both the arrays.

Input Format

· The first line contains T, the number of test cases. Following T lines contain:

1. Line 1 contains N1, followed by N1 integers of the first array

2. Line 2 contains N2, followed by N2 integers of the second array

Output Format

The intersection of the arrays in a single line

Example

Input:

1

3 10 17 57

6 2 7 10 15 57 246

Output:

10 57

Input:

1

6 1 2 3 4 5 6

2 1 6

Output:

1 6

**For example:**

| Input | Result |
|---|---|
| 1<br>3 10 17 57<br>6<br>2 7 10 15 57 246 | 10 57 |

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

void findIntersection(int *arr1, int n1, int *arr2, int n2) {
    int i = 0, j = 0;
    int first = 1; // Flag to handle spaces nicely between output elements

    while (i < n1 && j < n2) {
        if (arr1[i] == arr2[j]) {
            if (!first) {
                printf(" ");
            }
            printf("%d", arr1[i]);
            first = 0;
            i++;
            j++;
        } else if (arr1[i] < arr2[j]) {
            i++;
        } else {
            j++;
        }
    }
    printf("\n");
}

int main() {
    int t;
    // Read number of test cases
    if (scanf("%d", &t) == 1) {
```

```
    if (scanf("%d", &t) == 1) {
        while (t--) {
            int n1;
            scanf("%d", &n1);
            int *arr1 = (int *)malloc(n1 * sizeof(int));
            for (int i = 0; i < n1; i++) {
                scanf("%d", &arr1[i]);
            }

            int n2;
            scanf("%d", &n2);
            int *arr2 = (int *)malloc(n2 * sizeof(int));
            for (int i = 0; i < n2; i++) {
                scanf("%d", &arr2[i]);
            }

            // Find and print intersection for the current test case
            findIntersection(arr1, n1, arr2, n2);

            // Free allocated memory for the current test case
            free(arr1);
            free(arr2);
        }
    }
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 1<br>3 10 17 57<br>6<br>2 7 10 15 57 246 | 10 57 | 10 57 | ✔ |
| ✔ | 1<br>6 1 2 3 4 5 6<br>2<br>1 6 | 1 6 | 1 6 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Abdul Hameed A 251001003-2025-IT

A2

| | |
|---|---|
| **Started on** | Thursday, 24 September 2026, 10:25 AM |
| **State** | Finished |
| **Completed on** | Thursday, 24 September 2026, 10:26 AM |
| **Time taken** | 40 secs |
| **Marks** | 1.00/1.00 |
| **Grade** | **10.00** out of 10.00 (**100**%) |

**Question 1** | Correct Mark 1.00 out of 1.00

Find the intersection of two sorted arrays.

OR in other words,

Given 2 sorted arrays, find all the elements which occur in both the arrays.

Input Format

· The first line contains T, the number of test cases. Following T lines contain:

1. Line 1 contains N1, followed by N1 integers of the first array

2. Line 2 contains N2, followed by N2 integers of the second array

Output Format

The intersection of the arrays in a single line

Example

Input:

1

3 10 17 57

6 2 7 10 15 57 246

Output:

10 57

Input:

1

6 1 2 3 4 5 6

2 1 6

Output:

1 6

**For example:**

| Input | Result |
|---|---|
| 1<br>3 10 17 57<br>6<br>2 7 10 15 57 246 | 10 57 |

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

void findIntersection(int *arr1, int n1, int *arr2, int n2) {
    int i = 0, j = 0;
    int first = 1; // Flag to manage spaces between printed output elements

    while (i < n1 && j < n2) {
        if (arr1[i] == arr2[j]) {
            if (!first) {
                printf(" ");
            }
            printf("%d", arr1[i]);
            first = 0;
            i++;
            j++;
        } else if (arr1[i] < arr2[j]) {
            i++;
        } else {
            j++;
        }
    }
    printf("\n");
}

int main() {
    int t;
    // Read the number of test cases
    if (scanf("%d", &t) == 1) {
```

```
    if (scanf("%d", &t) == 1) {
        while (t--) {
            int n1;
            scanf("%d", &n1);
            int *arr1 = (int *)malloc(n1 * sizeof(int));
            for (int i = 0; i < n1; i++) {
                scanf("%d", &arr1[i]);
            }

            int n2;
            scanf("%d", &n2);
            int *arr2 = (int *)malloc(n2 * sizeof(int));
            for (int i = 0; i < n2; i++) {
                scanf("%d", &arr2[i]);
            }

            // Find and print the intersection for the current test case
            findIntersection(arr1, n1, arr2, n2);

            // Free allocated memory for this test case
            free(arr1);
            free(arr2);
        }
    }
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 1<br>3 10 17 57<br>6<br>2 7 10 15 57 246 | 10 57 | 10 57 | ✔ |
| ✔ | 1<br>6 1 2 3 4 5 6<br>2<br>1 6 | 1 6 | 1 6 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Abdul Hameed A 251001003-2025-IT

A2

| | |
|---|---|
| **Started on** | Thursday, 24 September 2026, 10:26 AM |
| **State** | Finished |
| **Completed on** | Thursday, 24 September 2026, 10:27 AM |
| **Time taken** | 49 secs |
| **Marks** | 1.00/1.00 |
| **Grade** | **10.00** out of 10.00 (**100**%) |

**Question 1** | Correct Mark 1.00 out of 1.00

Given an array A of sorted integers and another non negative integer k, find if there exists 2 indices i and j such that A[j] - A[i] = k, i != j.

Input Format:

First Line n - Number of elements in an array

Next n Lines - N elements in the array

k - Non - Negative Integer

Output Format:

1 - If pair exists

0 - If no pair exists

Explanation for the given Sample Testcase:

YES as 5 - 1 = 4

So Return 1.

**For example:**

| Input | Result |
|---|---|
| 3<br>1 3 5<br>4 | 1 |

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

int hasPairWithDifference(int *A, int n, int k) {
    if (n < 2) return 0;

    // Since k is non-negative, we can handle absolute difference or direct difference.
    // The problem specifies A[j] - A[i] = k with i != j.
    // Since A is sorted, we can set i = 0 and j = 1.
    int i = 0, j = 1;

    while (i < n && j < n) {
        if (i == j) {
            j++;
            continue;
        }

        int diff = A[j] - A[i];

        if (diff == k) {
            return 1; // Pair found
        } else if (diff < k) {
            j++; // We need a larger difference, so move the right pointer forward
        } else {
            i++; // We need a smaller difference, so move the left pointer forward
        }
    }

    return 0; // No such pair exists
}

int main() {
    int n;

    // Read the number of elements
    if (scanf("%d", &n) == 1) {
        int *A = (int *)malloc(n * sizeof(int));

        // Read the array elements
        for (int i = 0; i < n; i++) {
            scanf("%d", &A[i]);
        }

        int k;
        // Read the non-negative integer k
        scanf("%d", &k);

        // Output result
```

```
49          printf("%d\n", hasPairWithDifference(A, n, k));
50
51          free(A);
52      }
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 3<br>1 3 5<br>4 | 1 | 1 | ✔ |
| ✔ | 10<br>1 4 6 8 12 14 15 20 21 25<br>1 | 1 | 1 | ✔ |
| ✔ | 10<br>1 2 3 5 11 14 16 24 28 29<br>0 | 0 | 0 | ✔ |
| ✔ | 10<br>0 2 3 7 13 14 15 20 24 25<br>10 | 1 | 1 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Abdul Hameed A 251001003-2025-IT

A2

| | |
|---|---|
| **Started on** | Thursday, 24 September 2026, 10:27 AM |
| **State** | Finished |
| **Completed on** | Thursday, 24 September 2026, 10:28 AM |
| **Time taken** | 38 secs |
| **Marks** | 1.00/1.00 |
| **Grade** | **10.00** out of 10.00 (**100**%) |

**Question 1** | Correct Mark 1.00 out of 1.00

Given an array A of sorted integers and another non negative integer k, find if there exists 2 indices i and j such that A[j] - A[i] = k, i != j.

Input Format:

First Line n - Number of elements in an array

Next n Lines - N elements in the array

k - Non - Negative Integer

Output Format:

1 - If pair exists

0 - If no pair exists

Explanation for the given Sample Testcase:

YES as 5 - 1 = 4

So Return 1.

**For example:**

| Input | Result |
|---|---|
| 3<br>1 3 5<br>4 | 1 |

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

int hasPairWithDifference(int *A, int n, int k) {
    if (n < 2) return 0;

    int i = 0, j = 1;

    while (i < n && j < n) {
        // Indices must be distinct (i != j)
        if (i == j) {
            j++;
            continue;
        }

        int diff = A[j] - A[i];

        if (diff == k) {
            return 1; // Pair found
        } else if (diff < k) {
            j++; // Difference is too small, increase the upper pointer
        } else {
            i++; // Difference is too large, increase the lower pointer
        }
    }

    return 0; // No such pair exists
}

int main() {
    int n;

    // Read number of elements
    if (scanf("%d", &n) == 1) {
        int *A = (int *)malloc(n * sizeof(int));

        // Read the n elements (scanf automatically handles newlines or spaces)
        for (int i = 0; i < n; i++) {
            scanf("%d", &A[i]);
        }

        int k;
        // Read the non-negative integer k
        scanf("%d", &k);

        // Print the result (1 if exists, 0 otherwise)
        printf("%d\n", hasPairWithDifference(A, n, k));

```

```
49          free(A);
50      }
51
52      return 0;
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 3<br>1 3 5<br>4 | 1 | 1 | ✔ |
| ✔ | 10<br>1 4 6 8 12 14 15 20 21 25<br>1 | 1 | 1 | ✔ |
| ✔ | 10<br>1 2 3 5 11 14 16 24 28 29<br>0 | 0 | 0 | ✔ |
| ✔ | 10<br>0 2 3 7 13 14 15 20 24 25<br>10 | 1 | 1 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course